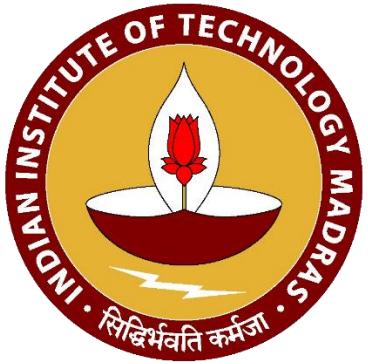
Python for Data Science
Web M. Tech Course DA5301W
Module 11: Debugging & Testing Code in Python

Dr. P.S Jayadev



Contents

- Intro to Debugging
- Types of Bugs in Python
- How to Perform Effective Debugging
- Testing in Python
- Types of Tests
- Pytest Framework

Intro to Debugging

- Process of detecting and resolving errors or unexpected behavior of a codebase
- Goes beyond fixing crashes—addresses correctness and assumptions
- Involves investigating code, data, environment, and logic
- Why Debugging matters in Data Science:
  - Data issues often remain hidden until late in the pipeline
    - Eg: Age read as string with characters other than numbers ('30 years') cannot be converted to float during analysis or modelling
  - Minor preprocessing mistakes cause major model performance drop

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # learns train mean/std
X_test_scaled  = scaler.transform(X_test)       # uses train mean/std
```

  - Silent failures occur – go unnoticed if not tested and debugged
  - Essential for reproducibility in notebooks and scripts

Types of Bugs in Python

Types of Bugs in Python

- Syntax Errors
- Runtime Errors
- Logical Errors
- Data Bugs
- Performance Bugs

Syntax Errors

- Definition: Errors in Python language structure which will not allow the code to run and occur during compilation
- Common causes:
  - Missing colon: Function definition, loops, conditional statements, etc.
  - Incorrect indentation
  - Unmatched parentheses/brackets
  - Misspelled keywords

Runtime Errors

- Definition: Errors that occur when the program is running.
- Runtime Errors can be categorized as follows:
  - Reference or Lookup Errors:
    - NameError: Happens when using a variable that hasn't been defined
    - KeyError: Accessing a missing column in a dataframe or dictionary key
    - AttributeError: Calling an attribute/method that doesn't exist on that object
    - IndexError: Accessing a non-existing index in a list, tuple, string, series or dataframe
  - Type & Value Errors:
    - TypeError: Happens when an operation is incompatible with type of the object(s) involved
    - ValueError: Happens when value is not corresponding to the type of the object

Runtime Errors

- Definition: Errors that occur when the program is running.
- Runtime Errors can be categorized as follows:
  - File or Module Errors:
    - FileNotFoundError: Happens when trying to read a file that does not exist
    - ModuleNotFoundError: Happens when importing a module that cannot be found
  - Computation Errors:
    - ZeroDivisionError: Happens when a number is divided by zero
    - MemoryError: Happens when operation needs more memory than available

Logical Errors

- Definition: Code runs without exceptions but produces wrong results
- Common causes
  - Wrong condition
    - Eg: if age > 18 or age < 65:
  - Incorrect formula
    - avg = df["salary"].sum() / len(df) # Wrong if salary column has NaNs
  - Formula applied on variables having incorrect units
  - Misplaced parentheses
  - Wrong variable used

Data Bugs

- Definition: Bugs caused by unexpected or malformed data and can happen silently; sometimes cause issues late in the pipeline
- Common causes
  - Wrong data type
    - When importing data all non-numerical columns are read as objects
  - Missing values
    - Eg: df[“x"] = df[“y"] / df[“z"]
    - If quantity column contains NaNs or zeros, no error is produced
    - ML modelling methods reject data with missing values
  - Unexpected categories
    - Ensure that categorical fields have same set of categories during training & testing models
  - Wrong merge keys or columns having different datatypes
    - df1["id"] = [101, 102, 103]
    - df2["id"] = ["101", "102", "103"] # merge will result in empty df

Performance Bugs

- Definition: Code runs correctly but too slowly or uses too much memory
- Common causes
  - Using loops instead of vectorization
  - Inefficient merges : merging on unindexed columns, merging inside a loop, etc.
  - Creating large intermediate DataFrames
- Tips to avoid above issues:
  - Ensure merge-key dtypes match
  - Select only required columns before merging
  - Don’t merge inside loops
  - Perform only index-based joins for huge data
  - Combine operations instead of making many copies
  - Delete intermediate dataframes which are not needed
  - Use chunking for huge files: reading/process data in small batches instead of loading everything at once

How to Perform Effective Debugging

Intro to Tracebacks

- A traceback is Python's error report that appears when an exception occurs
- Shows the sequence of function calls that led to the error
- Helps locate the exact file, line number, and code line where the program crashed
- Displays the exception type (e.g., KeyError, TypeError, ValueError)
- Reading from bottom to top tells you the root cause and how execution reached that point
- Essential tool for debugging in Python

Reading Tracebacks Effectively

- Identify file name, function calls, line number, and failing code line
- Interpret the exception type and error message carefully
- Follow the stack trace from bottom to top to understand the root cause
- Use traceback clues to form hypotheses about what went wrong
- Fix the error in way that it does not repeat in the future

Systematic Debugging Approach

- Isolate the code to the smallest failing snippet
- Add strategic logs/prints around suspect lines
- Inspect prior and intermediate values and shapes
- Test each hypothesis one at a time
- Fix, re-run, and validate correctness

Debugging in Notebooks and VSCode

- Debugging in Notebooks:
  - Use %pdb on: automatically enters debugger on errors
  - Use %debug after an exception: inspect state at failure
  - Supports ipdb-style commands:
    - p var (print variable)
    - u / d (up/down call stack)
    - l (list surrounding code)
  - Useful for inspecting data, shapes, intermediate values
- Debugging in VSCode or similar IDE:
  - Run in debug mode
  - Add breakpoints at which code execution stops
  - Run line by line or function by function
  - Inspect variables and values at each breakpoint

Testing in Python

Testing in Python

- Testing ensures that your code works correctly, reliably, consistently and efficiently
- Testing prevents following:
  - Runtime errors
    - Tests verify functions behave correctly for expected and edge cases
  - Logical Bugs
    - Code runs without errors but produces incorrect results
    - Tests check that outputs match expected values
  - Performance Issues
    - Tests can include performance checks including application and system level checks

Types of Tests

- Unit Tests
  - Test individual functions or classes in isolation
  - Ensure each small piece of code works as expected
  - Catch runtime bugs and logical bugs early

- Integration Tests
  - Test how multiple components work together
  - Validate interactions between modules
  - Catch errors due to misaligned assumptions between different parts of code
    - Eg: ML Pipeline from data pre-processing to validation

Types of Tests

- Performance Testing
  - Measure speed, memory usage, and scalability
  - Ensure performance stays within acceptable limits after code changes
  - Useful for data-heavy applications

- End to End Testing
  - Test the entire workflow from start to finish
  - Simulate real user or system behavior
  - Catch errors missed by unit and integration tests

Intro to Pytest

- Pytest is a popular, lightweight, and powerful testing framework for Python
- Makes it easy to write unit and integration tests
- Supports testing functions, classes and objects
- Works with simple python functions and assertions
- Automatically detects tests in your codebase
  - Files with test_*.py (or) *._test.py
  - Functions within files starting with test_
- Provides Clear, Readable Test Failures and Tracebacks

Key Features of Pytest

- Fixtures — Reusable Setup for Tests
  - Provide reusable objects for multiple tests
  - Useful for creating sample DataFrames, dummy models, etc.
  - Automatically run and provides return value before each test that requests them
  - Fixtures help you keep tests clean, tidy, and easy to understand.

- Parametrized Tests — Test Multiple Cases at one go
  - Run the same test function with multiple inputs and compare with target outputs
  - Useful for testing data preprocessing on edge cases
  - Produces separate test results for each set

Key Features of Pytest

- Approx — Safe Floating-Point Comparison
  - Handles rounding errors in numeric code
  - Ideal for ML, stats, normalization, probabilities

- Testing Exceptions using pytest.raises()
  - Ensure your code fails correctly or throw correct exceptions for invalid input

- Libraries for test reports: pytest-html (html report) and Allure (dashboard)
- Libraries for DS & ML related testing: pydantic and pandera

Type Hints in Python

- Form of syntax that allows developers to declare expected data types of variables, function parameters and return values
- Type hints improve clarity and catch runtime errors earlier, especially in DS/ML projects
- Improves documentation, autocomplete and IDE support
- typing library is built-in toolkit for writing type hints (for data structures like list, tuple, set, dict, etc.)

Data Validation using Pydantic and Pandera

Pydantic and Pandera

- Before data is used for ML or analytics, we must ensure:
  - Columns exist
  - Data types are correct (based on type hints)
  - Values fall within expected ranges
  - Missing values are handled
  - Bad data is caught early
- Pydantic and Pandera let us define rules for the data before it enters the processing pipeline

Features of Pydantic

- Pydantic contains a core class called BaseModel that is used to:
  - Validates the data
  - Converts types if needed and possible
  - Enforces range constraints or any other defined rules/validations
  - Raises human-readable errors when data is invalid
- Field() class is used to add rules or validations to individual data fields

Features of Pandera

- Pandera allows to:
  - Declare schemas for DataFrames
  - Validate data types, ranges, missing values, and conditions
- Schema Validation: Define required columns, column data types, rules for each column
- Has Built-in checks for standard rules and allows custom checks as well
- Missing value control: Choose to allow or disallow nulls
- Allows integration with pytest framework
- When validation/checks fails, Pandera shows:
  - which column failed
  - which rows failed
  - which rule was violated